

DBMS – PLSQL Lab1

Name: P. Supreeth Reddy

Roll No: AM.SC.U4AIE23142

1. Do all the questions discussed in the class

```
create table emp(eno int, ename varchar,sal numeric,dno int);
insert into emp values (1,'emp1',1000,1),(2,'emp2',2000,2),(3,'emp3',3000,2),(4,'emp4',2030,2)
insert into emp values (5,'emp5',7000,2);
```

1)

```
--1.
create or replace function oddoreven (a IN int) returns varchar as
$$
begin
    if a%2=1 then
        return 'odd';
    else
        return 'even';
    end if;
End;
$$
Language 'plpgsql';

select oddoreven(12);
```

	oddoreven character varying
1	even

2)

```
-- 2.Find Salary
create or replace function salFind(empno in emp.eno % type,sala out emp.sal%type) as
$$
begin
    select sal into sala from emp where eno=empno;
end;
$$
Language 'plpgsql';

select salFind(3);
```

	salfind numeric
1	3000

3)

```
-- 3.Find Number of Employees in Dept
create or replace function findNumberEmp(deptno emp.dno%type) returns int as
$$
declare
    count int;
begin
    select count(eno) into count from emp group by dno having emp.dno=deptno;
    return count;
end;
$$
Language 'plpgsql';

select findNumberEmp(2);
```

	findnumberemp integer	
1		4

4)

```
-- 4.Hike
create or replace function hike(empno emp.eno%type) returns void as
$$
declare
    avgsal int;
    cursal int;
begin
    select avg(sal) into avgsal from emp group by dno having dno in (select dno from emp where eno=empno);
    select sal into cursal from emp where empno=eno;

    if avgsal>cursal then
        update emp set sal=sal+0.1*cursal where eno=empno;
    end if;
end;
$$
Language 'plpgsql';

select hike(2);
select salFind(2);
```

	salfind numeric	
1	2200.0	

5)

```
-- 5.Find Department
create or replace function findDept(Empname emp.ename%type) returns emp.dno % type as
$$
declare
    deptno emp.dno%type;
begin
    select dno into deptno from emp where emp.ename=empname;
    return deptno;
end;
$$
Language 'plpgsql';

select findDept('emp3');
```

	finddept	
	integer	
1		2

2. Consider the following schema .

Emp(eno, ename, sal)

EMP_PROJ(eno, pno, prjt_hrs).

- Write a function ProjectLoad that returns the total project working hours for the given eno.
-
- Write a PL/SQL code to update the salary of an employee if the employee earn less than the average salary.

New salary is current sal + difference between current sal and average salary

```
create table empl(eno int primary key,ename varchar,sal numeric);
create table emp_proj(eno int,pno int,prhrs int,primary key(eno,pno),foreign key (eno) references empl);

insert into empl values (1,'emp1',1000),(2,'emp2',2300),(3,'emp3',3200);
insert into emp_proj values (1,1,12),(1,2,23),(2,1,2),(2,2,9),(3,1,4);
```

a)

```
create or replace function ProjectLoad (empno in empl.eno%type, hrs out int) as
$$
begin
    select sum(prhrs) into hrs from emp_proj group by eno having eno=empno;
end;
$$
Language 'plpgsql';

select(ProjectLoad(1));
```

	projectload	
	integer	
1		35

B,c)

```

create or replace function updateSal(empno empl.eno%type) returns void as
$$
declare
    avgsal int;
    cursal int;
begin
    select avg(sal) into avgsal from empl;
    select sal into cursal from empl where eno=empno;

    if cursal<avgsal then
        update empl set sal=sal+(avgsal-cursal) where empl.eno=empno;
    end if;
end;
$$
Language 'plpgsql';

select updateSal(1);
select * from empl where eno=1;

```

	eno [PK] integer	ename character varying	sal numeric
1	1	emp1	2167

Q3)

3. Consider the following tables

Item(ino, iname, unit_price)

Transaction(tr_no,ino,qty)

Write a function to accept an item no from the user. If transaction has been made for less than 2 times for that item(check from transaction table), delete the item from the Item table.

```

create table Item(ino int primary key, iname varchar, unit_price int);
create table Transactions(tr_no int primary key, ino int, qty int, foreign key (ino) references Item);

insert into Item values (1,'ItemA',100),(2,'ItemB',200),(3,'ItemC',300);
insert into Transactions values (101,1,5),(102,1,2),(103,2,1);

```

```

create or replace function checkAndDeleteItem(item_no int) returns void as
$$
declare
    trans_count int;
begin
    select count(*) into trans_count from Transactions where ino = item_no;

    if trans_count < 2 then
        delete from Transactions where ino = item_no;
        delete from Item where ino = item_no;
        raise notice 'Item deleted successfully';
    else
        raise notice 'Item has 2 or more transactions, not deleted';
    end if;
end;
$$
Language 'plpgsql';

select checkAndDeleteItem(2);
select * from Item;

```

	ino [PK] integer	iname character varying	unit_price integer
1	1	ItemA	100
2	3	ItemC	300

Q4)

4.

Consider a Bank database which includes the following tables.

ACCOUNTS(ac_no,br_no, cust_no,ac_type,bal)

BRANCHES(br_no, br_name, loc)

CUSTOMER(cno, cname, c_type)

- Write a function that accepts a threshold value and a customer number .
The program updates the c_type based on the threshold value. Ie. If balance> threshold then class A, else class B.
- Write a function called CloseBranch that takes two arguments (the branch to be closed and the branch to take over the accounts) and transfers all accounts at the closing branch to the new branch and removes the closing branch.
- Write a function that implements a "safe" withdrawal operation, that only permits a withdraw if there sufficient funds in the account to cover it.

```

create table branches(br_no int primary key, br_name varchar, loc varchar);
create table customer(cno int primary key, cname varchar, c_type varchar);
create table account(ac_no int primary key, br_no int, cust_no int, ac_type varchar, bal int,
foreign key (br_no) references branches, foreign key (cust_no) references customer);

insert into branches values (10, 'Main', 'CityA'), (20, 'Sub', 'CityB');
insert into customer values (1, 'Cust1', 'B'), (2, 'Cust2', 'B');
insert into account values (1001, 10, 1, 'Savings', 5000), (1002, 20, 2, 'Current', 2000);

```

A)

```

-- a) Update c_type based on threshold
create or replace function updateCustomerType(threshold int,cust_id int) returns void as
$$
declare
    total_bal int;
begin
    select sum(bal) into total_bal from account where cust_no = cust_id;

    if total_bal>threshold then
        update customer set c_type = 'A' where cno = cust_id;
    else
        update customer set c_type = 'B' where cno = cust_id;
    end if;
end;
$$
Language 'plpgsql';

select updateCustomerType(3000, 1);
select * from customer where cno=1;

```

	cno [PK] integer	cname character varying	c_type character varying
1	1	Cust1	A

b)

```

create or replace function CloseBranch(close_br_id int, takeover_br_id int) returns void as
$$
begin
    update account set br_no = takeover_br_id where br_no = close_br_id;
    delete from branches where br_no = close_br_id;
end;
$$
Language 'plpgsql';

select CloseBranch(20, 10);
select * from account;

```

	ac_no [PK] integer	br_no integer	cust_no integer	ac_type character varying	bal integer
1	1001	10	1	Savings	5000
2	1002	10	2	Current	2000

c)

```

create or replace function safeWithdraw(acc_id int,amount int) returns void as
$$
declare
    current_bal int;
begin
    select bal into current_bal from account where ac_no=acc_id;

    if current_bal >= amount then
        update account set bal = bal - amount where ac_no=acc_id;
        raise notice 'Withdrawal successful';
    else
        raise notice 'Insufficient funds';
    end if;
end;
$$
Language 'plpgsql';

select safeWithdraw(1001, 1000);
select * from account where ac_no=1001;

```

	ac_no [PK] integer 	br_no integer 	cust_no integer 	ac_type character varying 	bal integer 
1	1001	10	1	Savings	4000